

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 6

Networking dan REST API

Mengambil Data dari Internet dengan Ktor Client

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026

OUTLINE PERTEMUAN

01

Review Minggu Lalu

Navigasi Antar Layar

02

HTTP dan REST API

Konsep dasar web services

03

Ktor Client

HTTP client untuk KMP

04

JSON Serialization

Parsing dengan Kotlinx Serialization

05

Repository Pattern

Abstraksi data layer

06

Loading dan Error States

Handle UI states

CAPAIAN PEMBELAJARAN

CPMK0501

Mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak mobile multiplatform.

Indikator: Mampu mengintegrasikan aplikasi dengan REST API menggunakan Ktor Client

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami konsep HTTP dan REST API
- ✓ Menggunakan Ktor Client untuk HTTP requests
- ✓ Melakukan JSON serialization/deserialization
- ✓ Menerapkan Repository pattern untuk data layer
- ✓ Mengelola loading, success, dan error states

REVIEW MINGGU LALU

Pertemuan 5: Navigasi Antar Layar

Navigation Component	NavHost, NavController, Routes
Basic Navigation	navigate() dan popBackStack()
Passing Arguments	Required dan Optional arguments
Bottom Navigation	NavigationBar dengan tabs
Navigation Drawer	Side menu navigation

? Pertanyaan: Bagaimana cara mengambil data dari server/internet?

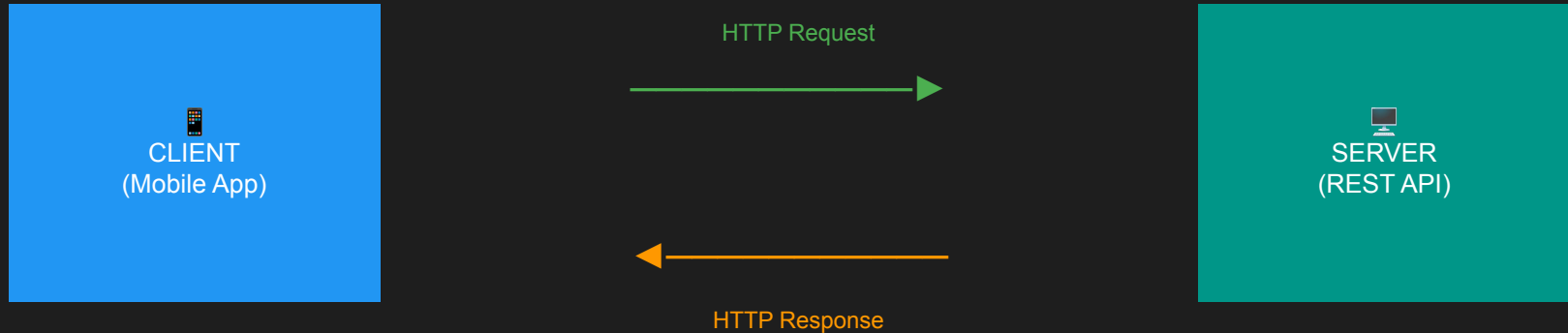
BAGIAN 1

HTTP DAN REST API

Konsep dasar komunikasi client-server

APA ITU HTTP?

HTTP (HyperText Transfer Protocol) adalah protokol komunikasi yang digunakan untuk transfer data di web. HTTP bekerja dengan model request-response: client mengirim request, server mengembalikan response.



HTTP METHODS (CRUD)

GET

Mengambil data dari server

GET /api/notes → Daftar semua notes

POST

Membuat data baru di server

POST /api/notes → Buat note baru

PUT

Update seluruh data yang ada

PUT /api/notes/1 → Update note id=1

PATCH

Update sebagian data

PATCH /api/notes/1 → Update partial

DELETE

Hapus data dari server

DELETE /api/notes/1 → Hapus note id=1

CRUD: Create (POST), Read (GET), Update (PUT/PATCH), Delete (DELETE)

APA ITU REST API?

REST (Representational State Transfer) adalah architectural style untuk web services. REST API menggunakan HTTP methods dan URL untuk mengakses resources. Data biasanya dikembalikan dalam format JSON.

Contoh REST API Endpoints:

Base URL: `https://api.example.com`

GET	<code>/users</code>	→ Semua users
GET	<code>/users/1</code>	→ User dengan id=1
POST	<code>/users</code>	→ Buat user baru
PUT	<code>/users/1</code>	→ Update user id=1
DELETE	<code>/users/1</code>	→ Hapus user id=1
GET	<code>/users/1/posts</code>	→ Semua posts dari user id=1

JSON FORMAT

JSON (JavaScript Object Notation) adalah format data ringan untuk pertukaran data:

Single Object

```
{
  "id": 1,
  "title": "Learn Kotlin",
  "content": "Kotlin is awesome!",
  "isCompleted": false
}
```

Array of Objects

```
[
  { "id": 1, "title": "Note 1" },
  { "id": 2, "title": "Note 2" },
  { "id": 3, "title": "Note 3" }
]
```

JSON Data Types:

- String: "text"
- Number: 123, 45.67
- Boolean: true, false
- Array: [1, 2, 3]
- Object: {"key": "value"}
- null

BAGIAN 2

KTOR CLIENT

HTTP Client untuk Kotlin Multiplatform

MENGAPA KTOR CLIENT?

Ktor adalah framework HTTP dari JetBrains yang mendukung Kotlin Multiplatform. Ktor Client memungkinkan kita melakukan HTTP requests dari Android, iOS, Desktop dengan satu codebase.

Keuntungan Ktor Client:

- | | |
|------------------------|--|
| ✓ Kotlin-first | Native Kotlin dengan Coroutines dan Flow |
| ✓ Multiplatform | Satu codebase untuk semua platform |
| ✓ Lightweight | Hanya include fitur yang dibutuhkan |
| ✓ Extensible | Plugin system untuk fitur tambahan |
| ✓ Type-safe | Serialization dengan Kotlinx Serialization |

SETUP DEPENDENCIES

build.gradle.kts

```
val ktorVersion = "2.3.7"

kotlin {
    sourceSets {
        commonMain.dependencies {
            // Ktor Client Core
            implementation("io.ktor:ktor-client-core:$ktorVersion")
            // Content Negotiation (untuk JSON)
            implementation("io.ktor:ktor-client-content-negotiation:$ktorVersion")
            // Kotlinx Serialization
            implementation("io.ktor:ktor-serialization-kotlinx-json:$ktorVersion")
            // Logging (optional)
            implementation("io.ktor:ktor-client-logging:$ktorVersion")
        }
        androidMain.dependencies {
            implementation("io.ktor:ktor-client-android:$ktorVersion")
        }
        iosMain.dependencies {
            implementation("io.ktor:ktor-client-darwin:$ktorVersion")
        }
    }
}
```

MEMBUAT HTTP CLIENT

HttpClientFactory.kt

```
import io.ktor.client.*
import io.ktor.client.plugins.contentnegotiation.*
import io.ktor.client.plugins.logging.*
import io.ktor.serialization.kotlinx.json.*
import kotlinx.serialization.json.Json

object HttpClientFactory {
    fun create(): HttpClient {
        return HttpClient {
            install(ContentNegotiation) {
                json(Json {
                    prettyPrint = true
                    isLenient = true
                    ignoreUnknownKeys = true // Abaikan field tidak dikenal
                })
            }
            install(Logging) {
                level = LogLevel.BODY
            }
        }
    }
}
```

GET REQUEST

GET Request

```
import io.ktor.client.call.*
import io.ktor.client.request.*

@Serializable
data class Note(
    val id: Int,
    val title: String,
    val content: String
)

class NoteApi(private val client: HttpClient) {
    private val baseUrl = "https://api.example.com"

    // GET semua notes
    suspend fun getAllNotes(): List<Note> {
        return client.get("$baseUrl/notes").body()
    }

    // GET note by ID
    suspend fun getNoteById(id: Int): Note {
        return client.get("$baseUrl/notes/$id").body()
    }
}
```

POST REQUEST

POST Request

```
import io.ktor.client.request.*
import io.ktor.http.*

@Serializable
data class CreateNoteRequest(
    val title: String,
    val content: String
)

class NoteApi(private val client: HttpClient) {

    // POST - Create new note
    suspend fun createNote(title: String, content: String): Note {
        return client.post("$baseUrl/notes") {
            contentType(ContentType.Application.Json)
            setBody(CreateNoteRequest(title, content))
        }.body()
    }
}
```

PUT DAN DELETE REQUEST

PUT & DELETE

```
class NoteApi(private val client: HttpClient) {  
  
    // PUT - Update note  
    suspend fun updateNote(id: Int, title: String, content: String): Note {  
        return client.put("$baseUrl/notes/$id") {  
            contentType(ContentType.Application.Json)  
            setBody(CreateNoteRequest(title, content))  
        }.body()  
    }  
  
    // PATCH - Partial update  
    suspend fun patchNote(id: Int, updates: Map<String, Any>): Note {  
        return client.patch("$baseUrl/notes/$id") {  
            contentType(ContentType.Application.Json)  
            setBody(updates)  
        }.body()  
    }  
  
    // DELETE - Hapus note  
    suspend fun deleteNote(id: Int): Boolean {  
        val response = client.delete("$baseUrl/notes/$id")  
        return response.status.isSuccess()  
    }  
}
```

BAGIAN 3

KOTLINX SERIALIZATION

Parsing JSON ke Data Class

KOTLINX SERIALIZATION

Kotlinx Serialization adalah library official dari Kotlin untuk serialize/deserialize data. Dengan `@Serializable` annotation, kita bisa convert JSON ke data class dan sebaliknya.

JSON Response

```
{
  "id": 1,
  "title": "My Note",
  "content": "Hello World",
  "created_at": "2024-01-15"
}
```



Data Class

```
@Serializable
data class Note(
    val id: Int,
    val title: String,
    val content: String,
    @SerializedName("created_at")
    val createdAt: String
)
```

Key Annotations:

`@Serializable` → Tandai class untuk serialization
`@SerializedName("name")` → Custom nama field di JSON
`@Transient` → Field tidak di-serialize

NESTED OBJECTS DAN LISTS

Complex JSON

```
// Nested object
@Serializable
data class User(
    val id: Int,
    val name: String,
    val email: String,
    val address: Address,    // Nested object
    val posts: List<Post>    // List of objects
)

@Serializable
data class Address(
    val street: String,
    val city: String,
    val zipCode: String
)

// API Response wrapper
@Serializable
data class ApiResponse<T>(
    val status: String,
    val message: String,
    val data: T              // Generic data
)
```

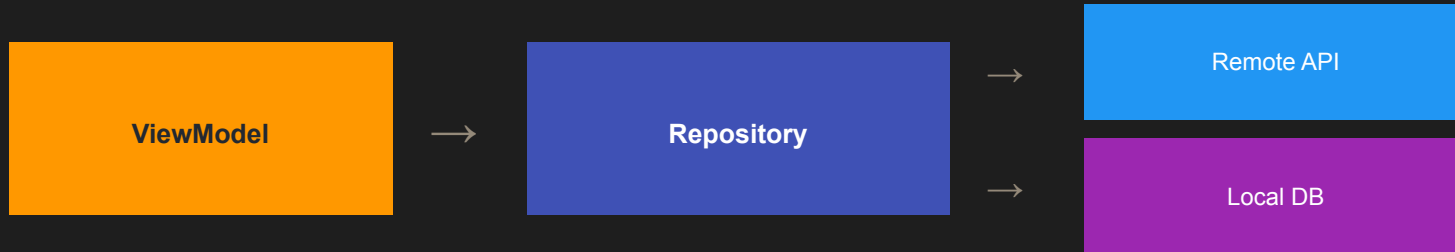
BAGIAN 4

REPOSITORY PATTERN

Abstraksi Data Layer

REPOSITORY PATTERN

Repository adalah layer yang memisahkan data source (API, database) dari business logic. ViewModel tidak perlu tahu dari mana data berasal - cukup minta ke Repository.



Benefits: • Single source of truth • Easy to test • Easy to swap data sources

REPOSITORY IMPLEMENTATION

NoteRepository.kt

```
class NoteRepository(private val client: HttpClient) {
    private val baseUrl = "https://jsonplaceholder.typicode.com"

    suspend fun getNotes(): Result<List<Note>> {
        return try {
            val notes: List<Note> = client.get("$baseUrl/posts").body()
            Result.success(notes)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    suspend fun getNoteById(id: Int): Result<Note> {
        return try {
            val note: Note = client.get("$baseUrl/posts/$id").body()
            Result.success(note)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    suspend fun createNote(title: String, body: String): Result<Note> {
        return try {
            val note: Note = client.post("$baseUrl/posts") {
                contentType(ContentType.Application.Json)
                setBody(mapOf("title" to title, "body" to body))
            }.body()
            Result.success(note)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

BAGIAN 5

LOADING DAN ERROR STATES

Mengelola UI berdasarkan state

UI STATE UNTUK NETWORKING

NotesUiState.kt

```
// Sealed class untuk UI State
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val message: String) : UiState<Nothing>()
}

// Atau gunakan data class dengan nullable
data class NotesUiState(
    val notes: List<Note> = emptyList(),
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)

// Helper extension untuk cek state
fun <T> UiState<T>.isLoading() = this is UiState.Loading
fun <T> UiState<T>.isSuccess() = this is UiState.Success
fun <T> UiState<T>.isError() = this is UiState.Error

// Get data safely
fun <T> UiState<T>.getOrNull(): T? = (this as? UiState.Success)?.data
```

VIEWMODEL DENGAN NETWORKING

NotesViewModel.kt

```
class NotesViewModel(private val repository: NoteRepository) : ViewModel() {

    private val _uiState = MutableStateFlow<UiState<List<Note>>>>(UiState.Loading)
    val uiState: StateFlow<UiState<List<Note>>>> = _uiState.asStateFlow()

    init {
        loadNotes()
    }

    fun loadNotes() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading

            repository.getNotes()
                .onSuccess { notes ->
                    _uiState.value = UiState.Success(notes)
                }
                .onFailure { error ->
                    _uiState.value = UiState.Error(
                        error.message ?: "Unknown error occurred"
                    )
                }
        }
    }

    fun refresh() { loadNotes() }
}
```

UI BERDASARKAN STATE

NotesScreen.kt

```
@Composable
fun NotesScreen(viewModel: NotesViewModel) {
    val uiState by viewModel.uiState.collectAsState()

    when (val state = uiState) {
        is UiState.Loading -> {
            Box(modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
        }

        is UiState.Success -> {
            LazyColumn {
                items(state.data) { note ->
                    NoteItem(note = note)
                }
            }
        }

        is UiState.Error -> {
            Column(modifier.fillMaxSize(),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center) {
                Text("Error: ${state.message}", color = Color.Red)
                Button(onClick = { viewModel.refresh() }) {
                    Text("Retry")
                }
            }
        }
    }
}
```

BAGIAN 6

HANDS-ON PRACTICE

3 Latihan Praktik

LATIHAN 1: FETCH DATA DARI API

Ambil data dari JSONPlaceholder API

Template Code

```
// API: jsonplaceholder.typicode.com/posts

@Serializable
data class Post(
    val id: Int,
    val userId: Int,
    val title: String,
    val body: String
)

class PostRepository(private val client: HttpClient) {
    suspend fun getPosts(): List<Post> {
        // TODO: Implement GET request
    }
}
```

Checklist:

- ☐ Setup Ktor Client
- ☐ Buat data class Post
- ☐ Implement repository
- ☐ GET request ke API
- ☐ Parse JSON response
- ☐ Tampilkan di LazyColumn



Waktu: 20 menit

LATIHAN 2: LOADING DAN ERROR STATES

Implementasi UI untuk berbagai states

Template Code

```
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val message: String) : UiState<Nothing>()
}

@Composable
fun PostsScreen(viewModel: PostsViewModel) {
    val uiState by viewModel.uiState.collectAsState()

    when (val state = uiState) {
        is UiState.Loading -> { /* TODO */ }
        is UiState.Success -> { /* TODO */ }
        is UiState.Error -> { /* TODO */ }
    }
}
```

Checklist:

- ☐ UiState sealed class
- ☐ ViewModel dengan StateFlow
- ☐ Loading: CircularProgressIndicator
- ☐ Success: LazyColumn
- ☐ Error: Text + Retry button
- ☐ Test dengan airplane mode



Waktu: 20 menit

LATIHAN 3: POST DATA KE API

Membuat data baru dengan POST request

Template Code

```
// Repository
suspend fun createPost(title: String, body: String): Result<Post> {
    return try {
        val post: Post = client.post("$baseUrl/posts") {
            contentType(ContentType.Application.Json)
            setBody(mapOf(
                "title" to title,
                "body" to body,
                "userId" to 1
            ))
        }.body()
        Result.success(post)
    } catch (e: Exception) {
        Result.failure(e)
    }
}
```

Checklist:

- ☐ Form dengan TextField
- ☐ POST request
- ☐ Set content type JSON
- ☐ Send body data
- ☐ Handle response
- ☐ Show success message
- ☐ Update list

 Waktu: 25 menit

TROUBLESHOOTING

✗ No internet connection

✓ Wrap dalam try-catch, tampilkan error message

✗ JSON parse error

✓ Gunakan ignoreUnknownKeys = true, periksa field names

✗ SSL/TLS error

✓ Pastikan URL menggunakan HTTPS

✗ Timeout exception

✓ Set timeout di HttpClient configuration

BEST PRACTICES

✓ Repository Pattern

Pisahkan logic API dari ViewModel

✓ Handle All States

Loading, Success, Error - jangan ada yang terlewat

✓ Use Result/Either

Wrap response untuk handle success dan failure

✓ Timeout Configuration

Set timeout untuk mencegah request hang

✓ Error Messages

Tampilkan pesan error yang user-friendly

✓ Retry Mechanism

Berikan opsi retry untuk error yang recoverable

RANGKUMAN PERTEMUAN 6

HTTP/REST

Protokol komunikasi client-server dengan methods CRUD

JSON

Format data ringan untuk pertukaran data

Ktor Client

HTTP client untuk Kotlin Multiplatform

Serialization

@Serializable untuk parsing JSON ke data class

Repository

Pattern untuk memisahkan API logic

UI States

Loading, Success, Error untuk reactive UI

 **Networking = Menghubungkan aplikasi dengan dunia luar!**

TUGAS PRAKTIKUM MINGGU 6

Bobot: 4% | Deadline: Sebelum Pertemuan 7

Deskripsi Tugas:

Buat aplikasi "News Reader" dengan fitur:

1. Fetch berita dari public API (NewsAPI atau JSONPlaceholder)
2. Tampilkan list artikel dengan title, description, dan image
3. Detail screen saat artikel di-klik
4. Pull to refresh functionality
5. Proper loading, success, dan error states
6. Repository pattern untuk API calls

Format Pengumpulan:

- Push ke GitHub repository (branch: week-6)
- README: API yang digunakan, screenshot semua states
- Video demo (30 detik) menunjukkan loading, success, error, dan refresh

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
API Integration	25%	Ktor Client setup, successful API calls
Data Parsing	20%	Kotlinx Serialization, proper data classes
UI States	25%	Loading, Success, Error handled properly
Architecture	20%	Repository pattern, clean separation
Code Quality	10%	Clean code, documentation
 Bonus (+10%) Offline caching dengan local storage		 Penalti Terlambat: -10%/hari, Plagiat: 0

SUMBER PUSTAKA

Dokumentasi

Ktor Client

ktor.io/docs/client.html

Dokumentasi

Kotlinx Serialization

github.com/Kotlin/kotlinx.serialization

API

JSONPlaceholder

jsonplaceholder.typicode.com

API

NewsAPI

newsapi.org

Guide

REST API Design

restfulapi.net

Article

Ktor for KMP

ktor.io/docs/getting-started-ktor-client-multiplatform.html

PREVIEW MINGGU DEPAN

Pertemuan 7: Local Data Storage

DataStore

Key-value storage untuk preferences

SQLDelight

Type-safe SQL untuk KMP

Offline First

Cache data untuk offline access

Secure Storage

Menyimpan data sensitif

Persiapan:

- Selesaikan tugas News Reader dengan API
- Baca dokumentasi SQLDelight
- Pahami konsep offline-first architecture



SESI TANYA JAWAB

Ada pertanyaan tentang Networking dan REST API?

CHECKLIST SEBELUM PULANG

Pastikan Anda sudah:

- ☐ Memahami konsep HTTP dan REST API
- ☐ Bisa setup Ktor Client dengan dependencies
- ☐ Mampu melakukan GET dan POST requests
- ☐ Mengerti cara parsing JSON dengan Kotlinx Serialization
- ☐ Bisa implement Repository pattern
- ☐ Mampu handle Loading, Success, dan Error states
- ☐ Mengerti cara integrasi networking dengan ViewModel
- ☐ Mencatat deadline tugas News Reader

Terima Kasih

Pertemuan 6: Networking dan REST API

*"The best apps are not islands.
They connect to the world."*

- Mobile Development Wisdom

Selamat menghubungkan aplikasi Anda dengan dunia! 🌐